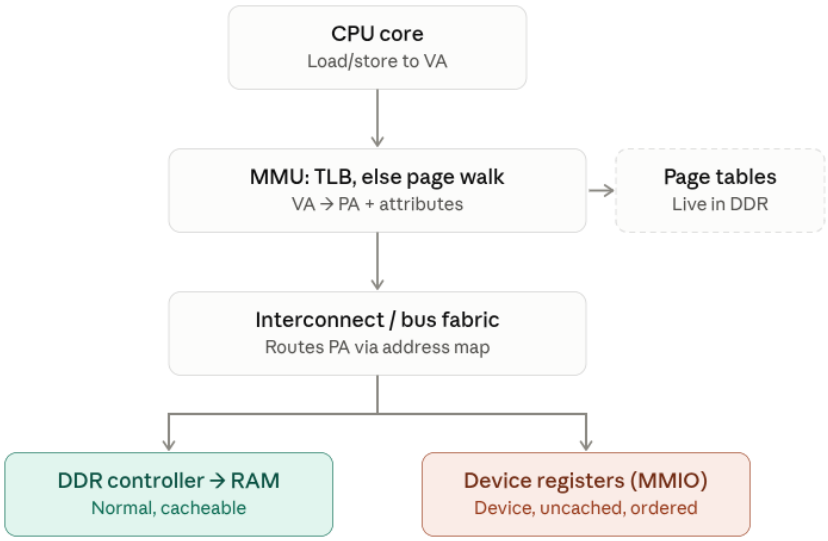
How mmu translates virtual to ddr and devices(MMIO)



Step 1: VA → PA (the MMU's job)

Take ARMv8-A with 4 KB pages and 48-bit VAs as a concrete example. The VA is sliced into fields:

VA[47:39] → L0 index VA[38:30] → L1 index
VA[29:21] → L2 index VA[20:12] → L3 index
VA[11:0] → page offset (copied unchanged into PA)

1. **TLB lookup first.** If the VA's page is cached in the TLB (hit), you get PA + attributes in ~1 cycle.
2. **On a miss, the hardware page-table walker runs.** It starts from the base register (TTBR0_EL1 for user space, TTBR1_EL1 for kernel; on x86 it's CR3), uses each index to pick an entry in each level, and ends at a leaf PTE.
3. **The leaf PTE holds** the physical frame number, permissions (R/W/X, user/kernel), and a memory-type index (AttrIndx, pointing into the MAIR_EL1 register).
4. **PA = frame number + page offset.** The result is cached in the TLB.

Note that the page tables themselves are stored in DDR, so a TLB miss costs several memory reads (cached in the data caches where possible).

Step 2: PA → DDR or device (the interconnect's job)

Every SoC has a fixed **physical address map**, set by the chip designers. For example, on QEMU's ARM virt machine, RAM starts at 0x4000_0000 and the PL011 UART sits at 0x0900_0000. When the CPU emits a PA onto the bus (AXI/CHI), the interconnect decoder checks which range it falls in:

- PA in the DRAM range → sent to the **DDR memory controller** → physical RAM.
- PA in a peripheral range → sent to that **device's register block** (UART, GPIO, PCIe BAR, etc.). Writing to 0x0900_0000 doesn't store data; it pushes a byte into the UART's TX FIFO.

So "MMIO" just means device registers are reached with ordinary load/store instructions at certain physical addresses.

Why the attributes matter

The MMU can't tell DDR from a device, so **the OS must tell it** via the PTE's memory type. This is the real difference between mapping RAM and mapping a device:

	DDR (Normal memory)	**MMIO (Device memory)**
Caching	Write-back cacheable	Uncached
Speculative reads	Allowed	Forbidden (a read can have side effects, e.g. popping a FIFO)
Reordering/merging	Allowed	Disallowed (ARM Device-nGnRE: no Gathering, no Reordering)
x86 equivalent	WB via PAT/MTRR	UC via PAT/MTRR

If you mistakenly map device registers as cacheable, writes may sit in cache and never reach the device, and status reads return stale values.

How Linux does it

- **DDR**: the kernel maps all RAM in its linear map as Normal memory, and user pages are mapped on demand at page-fault time.
- **MMIO**: drivers call ioremap(phys_addr, size), which creates a kernel VA mapping with Device attributes, and then access it with readl()/writel() (which also add the needed barriers).

The reverse direction: SMMU/IOMMU

When a *device* does DMA into DDR, it uses its own address (IOVA). An **SMMU** (ARM) or **IOMMU** (x86 VT-d) translates IOVA → PA with the same page-table idea, so devices can be isolated and given contiguous-looking buffers.

You can click any box in the diagram to dig deeper into that stage.

How does the CPU generate virtual addresses and select TTBR0 vs TTBR1?

Part 1: Where a virtual address comes from

The CPU doesn't "generate" a VA in any special way. A VA is just the 64-bit number an instruction computes as its memory address, *before* the MMU sees it. There are three sources:

1. **Load/store instructions**: the address-generation unit (AGU) computes base register + offset or index: LDR X0, [X1, #16] // VA = X1 + 16STR W2, [X3, X4, LSL #2] // VA = X3 + (X4 << 2)

2. **Instruction fetch**: the VA is the program counter (PC).
3. **Stack operations**: the VA comes from SP.

The values in those registers originate from the toolchain and OS. The linker places code and data at chosen VAs in the ELF file. The kernel's loader mmaps those segments at those VAs (shifted by ASLR). malloc returns VAs that the kernel handed out via mmap/brk. PC-relative instructions like ADRP + ADD build addresses relative to wherever the code was loaded. The program only ever deals in VAs; physical addresses never appear in user code.

One ARM-specific detail: if **Top Byte Ignore** (TCR_EL1.TBI0/TBI1) is enabled, bits [63:56] are ignored for translation. This frees them for pointer tags (MTE) or pointer authentication codes (PAC).

Part 2: How TTBR0 vs TTBR1 is chosen

At EL1/EL0, the 64-bit VA space is split into **two separate ranges**, each with its own page-table base register. The selection is made purely from **the address bits**, not from the privilege level.

TCR_EL1.T0SZ and T1SZ set each range's size. With the common setting T0SZ = T1SZ = 16 (48-bit VAs):

VA range	Upper bits	Translated with	Typical use
0x0000_0000_0000_0000 to 0x0000_FFFF_FFFF_FFFF	all 0s	TTBR0_EL1	User space (per process)
0xFFFF_0000_0000_0000 to 0xFFFF_FFFF_FFFF_FFFF	all 1s	TTBR1_EL1	Kernel (shared by all)
Everything in between	mixed	none	Translation fault

The hardware logic is roughly:

```
if (VA[55] == 0) {            // bit 55, so it works even with TBI
   if (VA[63:48] != 0x0000) fault();   // top bits must all be 0 (unless TBI)
   base = TTBR0_EL1;          // range size from T0SZ
} else {
   if (VA[63:48] != 0xFFFF) fault();   // top bits must all be 1
   base = TTBR1_EL1;          // range size from T1SZ
}
// then index into the tables at base using VA[47:12]
```

Bit 55 is used (rather than bit 63) so that selection still works when the top byte holds a tag.

Concrete examples on Linux arm64:

- A user pointer 0x0000_aaaa_dead_b000 has top bits 0, so it walks TTBR0_EL1 (this process's tables).
- A kernel pointer 0xffff_8000_1000_0000 has top bits 1, so it walks TTBR1_EL1 (kernel tables).
- 0x00ff_0000_0000_0000 lands in the hole and faults.

Why the split exists

Context switches are cheap. Each process gets its own TTBR0 tables, while the kernel's TTBR1 mapping stays the same for everyone. On a switch, the kernel just writes the new process's table address into TTBR0, along with an **ASID** (address space ID) in its upper bits. TLB entries are tagged with the ASID, so the old process's entries don't need flushing. Kernel entries are marked global (nG = 0), so they survive every switch.

Selection is by address, not privilege

Because the choice depends only on the VA:

- **The kernel (EL1) can access user addresses**, because they go through TTBR0. This is how copy_from_user() works. ARM's **PAN** (Privileged Access Never) blocks this unless the kernel explicitly allows it, to catch bugs and exploits.
- **User code (EL0) touching a kernel address** still selects TTBR1, but the PTE's access permission bits deny EL0, giving a permission fault. With **KPTI** (the Meltdown mitigation), Linux additionally keeps most kernel mappings out of the tables active while in user mode.
- TCR_EL1.EPD0/EPD1 can disable walks for either range entirely, making any TLB miss there fault.

Other exception levels and x86

EL2 normally has only TTBR0_EL2, but with VHE (HCR_EL2.E2H = 1) it gains TTBR1_EL2, so a host kernel can run at EL2 with the same split layout. EL3 has only TTBR0_EL3.

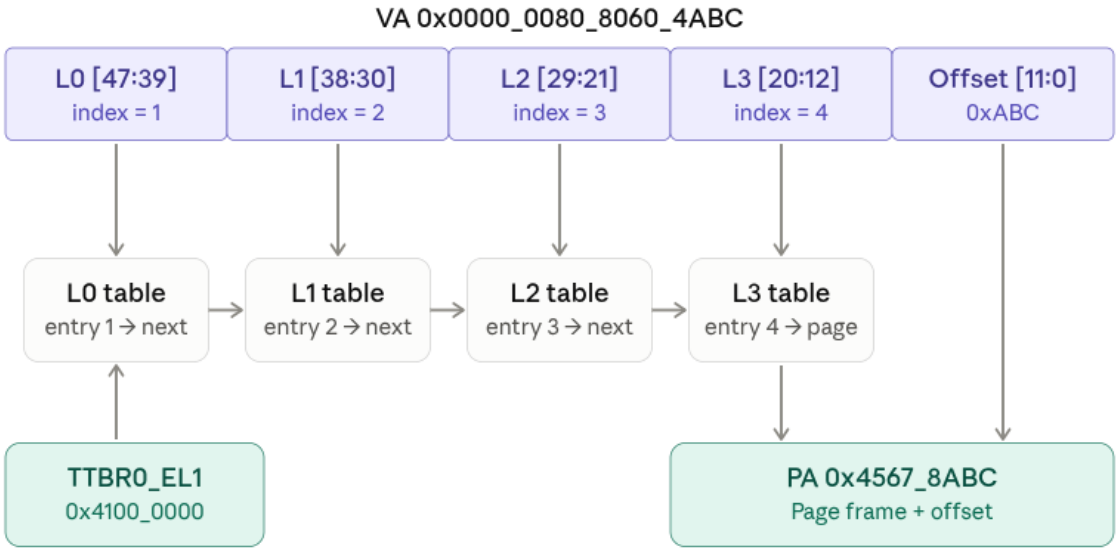
x86-64 has no hardware split: there is a single CR3, and the kernel occupies the upper half by convention. Instead, addresses must be **canonical** (bits 63:47 all equal), and a non-canonical address raises a #GP fault, which is the equivalent of ARM's hole in the middle.

Explain the TLB and multi-level page table walk in detail

A TLB miss triggers a page table walk, and the TLB exists to make sure that walk happens as rarely as possible. Here's the walk first, then the caching that hides it.

The page table walk

I'll use the standard ARMv8-A setup: 4 KB pages, 48-bit VAs, 4 levels (L0 to L3). As a worked example, let's translate **VA 0x0000_0080_8060_4ABC**.



Why the VA is sliced this way

Each table is exactly one 4 KB page. Each entry (descriptor) is 8 bytes, so a table holds 4096 / 8 = **512 entries**, which needs **9 bits** of index. Four levels × 9 bits + 12 offset bits = 48 bits of VA. The 12-bit offset is never translated; it's copied straight into the PA because it addresses a byte *within* the 4 KB page.

The walk, step by step

Each step computes entry_address = table_base + index × 8, reads the 8-byte descriptor from memory, and extracts the next table's base from it:

Step	Table base	Index	Entry address read	Descriptor says
L0	0x4100_0000 (from TTBR0)	1	0x4100_0008	Table → 0x4100_1000
L1	0x4100_1000	2	0x4100_1010	Table → 0x4100_2000
L2	0x4100_2000	3	0x4100_2018	Table → 0x4100_3000
L3	0x4100_3000	4	0x4100_3020	Page → frame 0x4567_8000 + attributes

Final PA = 0x4567_8000 | 0xABC = **0x4567_8ABC**.

Note that every address in the walk is **physical**. The walker can't use virtual addresses to find page tables, or it would recurse forever. That's why the kernel must know the physical location of every table it builds.

What's inside a descriptor

Bits [1:0] decide what kind of entry it is:

- x0 → **invalid**: translation fault.
- 11 at L0 to L2 → **table descriptor**: points to the next-level table.
- 01 at L1 or L2 → **block descriptor**: the walk stops early and maps a huge region directly (1 GB at L1, 2 MB at L2).
- 11 at L3 → **page descriptor**: maps a single 4 KB page.

A leaf (block or page) descriptor carries the output address in bits [47:12] plus the attributes:

Bits	Field	Meaning
[4:2]	AttrIndx	Index into MAIR_EL1: Normal cacheable vs Device memory
[7:6]	AP	Read/write permissions, EL0 access allowed or not
[9:8]	SH	Shareability (cache coherence domain)
10	AF	Access flag: set when the page has been used
11	nG	Not-global: if set, the TLB entry is tied to the current ASID
51	DBM	Dirty bit modifier (hardware dirty tracking)
52	Contiguous	Hint that 16 adjacent entries map a contiguous region
53, 54	PXN, UXN	Execute-never for kernel and user

Table descriptors can also carry hierarchical controls (APTable, XNTable, PXNTable) that restrict everything underneath them.

Block mappings (huge pages)

If the L2 descriptor were a block, the walk would stop there: VA[20:0] becomes a 21-bit offset into a **2 MB** region. That saves one memory read per walk, and more importantly, one TLB entry now covers 2 MB instead of 4 KB. Linux uses this for its linear map of RAM and for transparent huge pages.

When the walk fails

The walker raises a synchronous exception to EL1. ESR_EL1 records the fault type and the level where it happened; FAR_EL1 holds the faulting VA. The common types are:

- **Translation fault** (invalid descriptor at some level). Linux's page fault handler treats this as "not mapped yet": it allocates a page, writes the PTE, and retries the instruction. This is how demand paging works.
- **Access flag fault** when AF = 0, unless hardware AF management (TCR_EL1.HA) is enabled.
- **Permission fault**: for example, a write to a read-only page (this is how copy-on-write after fork() is triggered).

The TLB: avoiding walks

A full walk is **4 dependent memory reads** before the actual data access can even start. Doing that on every load and store would be ruinous, so the TLB caches finished translations.

What a TLB entry holds

Each entry maps a VA page → PA frame plus all the leaf attributes. It's tagged with:

- **ASID** (address space ID, from TTBR0's upper bits): lets entries from different processes coexist, so a context switch doesn't require a flush.
- **VMID** (when virtualization is in use): separates guest VMs.
- **Global flag**: kernel mappings (nG = 0) match regardless of ASID.
- **Page/block size**: one entry can cover 4 KB, 64 KB (contiguous), 2 MB or 1 GB.

The TLB hierarchy

Modern cores have two levels of TLB:

1. **L1 micro-TLBs**: separate instruction and data TLBs, small (tens of entries), fully associative, checked in parallel with the L1 cache lookup on every access.
2. **L2 unified TLB**: larger (typically a thousand or more entries), set-associative, a few cycles slower.

Only when both miss does the hardware walker run.

TLB reach = entries × page size. With 1,000 entries of 4 KB, that's only about 4 MB of memory covered. The same 1,000 entries of 2 MB blocks cover about 2 GB. This is why huge pages make such a difference for databases and large-memory workloads.

Making the walk itself cheaper

Even when a walk does happen, it's rarely 4 full trips to DDR:

- **Walk caches** (also called intermediate or partial-translation caches) store upper-level descriptors. For neighboring addresses, the L0 to L2 entries are identical, so the walker can often jump straight to the L3 read.
- **Data caches**: the walker's reads go through the normal cache hierarchy (if TCR_EL1.IRGN/ORGN mark the tables cacheable), so descriptors often come from L2 or L3 cache rather than DDR.

Keeping the TLB in sync (TLB maintenance)

The TLB is **not** automatically kept coherent with page table writes. If the kernel changes a PTE, the stale translation may still be cached. Software must invalidate it explicitly. The canonical arm64 sequence is:

```
STR   X1, [X0]        // write the new PTE
DSB   ISHST           // make the write visible to the walker
TLBI  VAE1IS, X2      // invalidate this VA (with ASID) on all cores
DSB   ISH             // wait for the invalidation to complete everywhere
ISB                   // resync this core's instruction stream
```

The IS (Inner Shareable) suffix makes the hardware **broadcast** the invalidation to every core. Other variants include TLBI ASIDE1IS (drop everything for one ASID) and TLBI VMALLE1IS (drop all EL1 entries).

Comparisons and extensions

x86-64 works almost identically: 4 levels (PML4 → PDPT → PD → PT), 9-bit indices, 4 KB pages with 2 MB and 1 GB huge pages, and CR3 as the root. PCID plays the role of ASID. Hardware always sets the Accessed and Dirty bits. The big difference is TLB shootdown: INVLPG only affects the local core, so the kernel traditionally sends inter-processor interrupts to make other cores flush, rather than relying on a hardware broadcast like ARM's TLBI …IS. (AMD has since added INVLPGB for broadcast invalidation.) With 5-level paging (LA57), x86 extends to 57-bit VAs; ARM similarly offers 52-bit VAs.

Virtualization makes walks two-dimensional. A guest's page tables translate guest VA → guest PA (stage 1), and the hypervisor's tables translate guest PA → host PA (stage 2). Every guest table pointer is itself a guest PA that needs a stage-2 walk. With 4 levels in each stage, a worst-case TLB miss costs up to **24 memory reads** ($5 \times 5 - 1$). This is why the TLB, walk caches, and huge pages matter even more inside VMs.

How are page tables laid out in memory and what is inside a PTE?

I covered the main PTE fields in the last answer, so here I'll focus on how the tables sit in physical memory, then give the complete PTE bit layout including the parts the OS uses for its own purposes.

How page tables are laid out in memory

Each table is just a physical page

With a 4 KB granule, every table at every level is exactly one 4 KB page of physical RAM, holding 512 eight-byte descriptors. Three properties follow from this:

1. **Tables are 4 KB aligned.** The low 12 bits of a table's address are always zero, which is why a descriptor only needs to store bits [47:12] of the next table's address. The freed low bits hold the type and attribute flags.
2. **Tables can be anywhere in RAM.** They are not contiguous and not in any special region. The kernel grabs them from the ordinary page allocator one at a time, as needed. An L1 table at 0x4100_1000 might point to an L2 table at 0x7a3c_2000.
3. **All pointers are physical.** TTBR and every table descriptor hold physical addresses, because the hardware walker reads physical memory directly.

The tree is sparse

A process's address space is mostly empty, so most of the tree never exists. Consider a typical arm64 Linux process:

```
Code/heap near   0x0000_aaaa_....  → L0 index 341
Stack/libs near  0x0000_ffff_....  → L0 index 511

L0 table (1 page): 512 entries, only 2 valid
 ├─ [341] → L1 table → a few L2 tables → L3 tables for code, data, heap
 └─ [511] → L1 table → a few L2 tables → L3 tables for libs, stack
(other 510 entries: invalid, no tables allocated beneath them)
```

A small process may need only around 10 to 20 table pages in total. Tables are allocated lazily: when a page fault hits a region with no L3 table yet, the kernel allocates one, links it into the L2 entry, then fills in the PTE.

Overhead check: mapping 1 GB fully with 4 KB pages needs 262,144 PTEs × 8 bytes = 2 MB of L3 tables, which is about 0.2% overhead. Mapping it with a single 1 GB block at L1 needs zero L2 or L3 tables.

Linux naming and organization

Linux uses generic names for the levels, which map onto arm64's 4-level layout as follows:

Linux name	arm64 level	Each entry covers
pgd (page global directory)	L0	512 GB
p4d	folded away (only used with 5 levels)	
pud (page upper directory)	L1	1 GB
pmd (page middle directory)	L2	2 MB
pte (page table entry)	L3	4 KB

Each process's mm_struct->pgd points to its L0 table, and on a context switch the kernel loads that table's physical address (plus the ASID) into TTBR0_EL1.

The kernel's own tables (rooted at swapper_pg_dir) are loaded into TTBR1_EL1 once and shared by every process. x86 has only one root (CR3), so instead each process's top-level table contains copies of the kernel's upper-half entries, all pointing to the *same* shared lower-level kernel tables.

How the kernel edits tables it can only reach physically

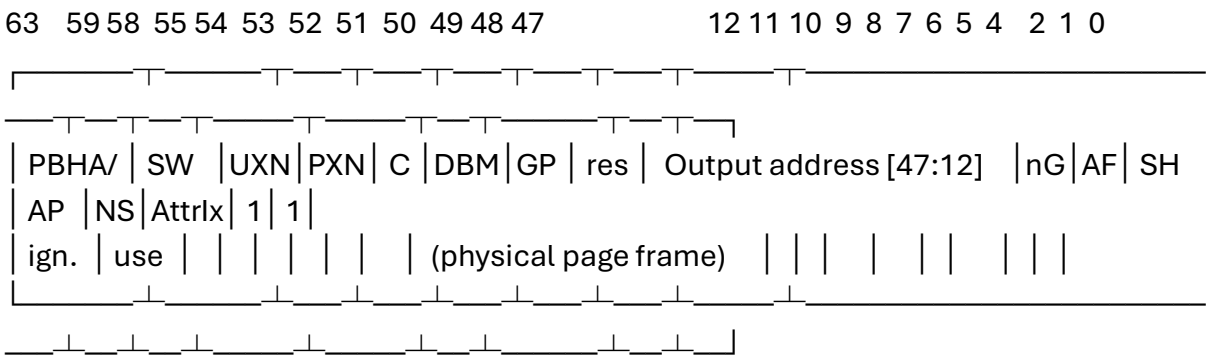
The walker uses physical addresses, but kernel code runs with the MMU on and needs virtual addresses to read and write table entries. Linux solves this with its **linear map**: all of RAM is mapped at a fixed offset in the kernel's address space, so converting is simple arithmetic:

```
void *va     = __va(table_phys);   // phys + fixed offset
phys_addr_t pa = __pa(table_va);   // the reverse
```

So the same table page has two "names": its physical address (used inside descriptors and TTBR) and its linear-map virtual address (used by kernel C code). Some other OSes

use a different trick called a **recursive (self-referencing) mapping**, where one top-level entry points back at the top-level table itself, making all tables appear at a fixed virtual window. Windows on x86 historically did this.

The complete PTE (arm64, 4 KB page at L3)



Beyond the fields I described last time, a few encodings are worth seeing in full.

AP[2:1] (bits 7:6), access permissions:

Value	Kernel (EL1)	User (EL0)
00	Read/write	No access
01	Read/write	Read/write
10	Read-only	No access
11	Read-only	Read-only

Execute permission is separate, via PXN (kernel) and UXN (user).

SH (bits 9:8), shareability: 00 non-shareable, 10 outer shareable, 11 inner shareable. Normal RAM on a multicore system is marked inner shareable so all cores' caches stay coherent for it.

AttrIndx (bits 4:2) and MAIR_EL1: the PTE doesn't store the memory type directly. It stores a 3-bit index into MAIR_EL1, a register holding up to 8 one-byte memory-type definitions that the OS sets at boot. A typical setup looks like:

MAIR slot	Encoding	Memory type	Used for
Attr0	0x00	Device-nGnRnE	Strictly ordered device registers
Attr1	0x04	Device-nGnRE	Normal MMIO (ioremap)

Attr2	0x44	Normal non-cacheable	Some DMA buffers, framebuffers
Attr3	0xFF	Normal write-back cacheable	Regular RAM

This indirection is what ties back to your original question: the *only* thing distinguishing a DDR mapping from an MMIO mapping in the PTE is which AttrIndx the OS wrote.

Bits the hardware ignores, which the OS uses

Software bits [58:55] are reserved for the OS. Linux uses them to track things like a software dirty flag and "special" mappings that have no normal struct page behind them.

Dirty tracking via DBM (bit 51): arm64 has no plain hardware "dirty" bit. Instead, Linux marks a writable but clean page as *read-only with DBM = 1*. On the first write, hardware (with FEAT_HAFDBS) flips it to writable automatically instead of faulting. The kernel then reads "writable" as meaning "dirty", which tells it the page must be written back before being reclaimed.

Invalid PTEs carry OS data. When bit 0 is clear, the hardware ignores the other 63 bits completely. Linux exploits this: when a page is swapped out, its PTE becomes invalid and the remaining bits store a **swap entry** (which swap device and the slot offset). The next access faults, and the fault handler decodes the entry to read the page back in.

For comparison: the x86-64 PTE

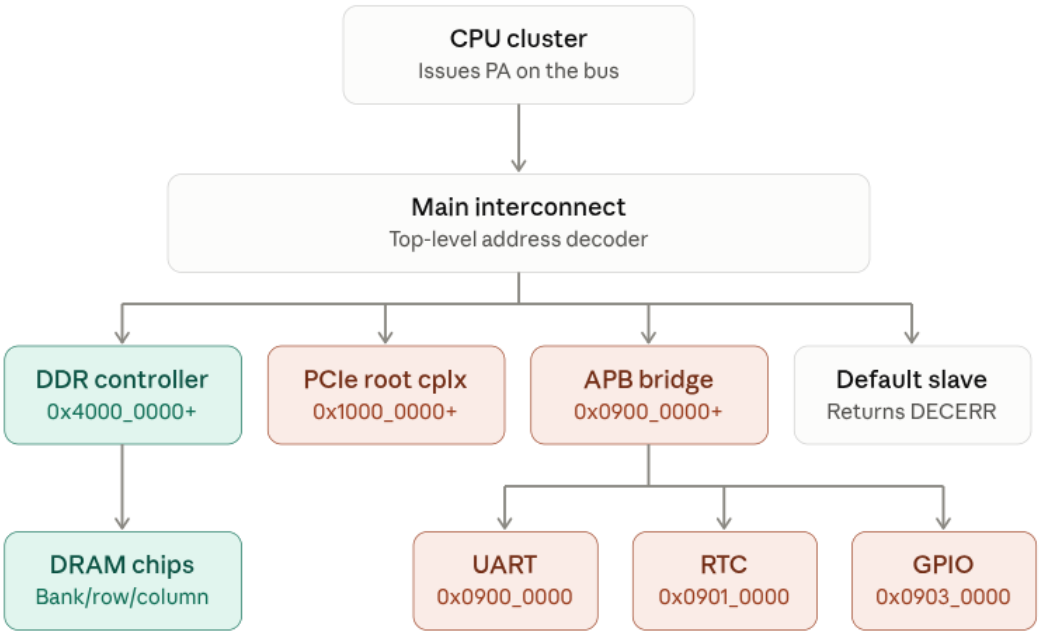
Bits	Field	Notes
0	P (present)	Equivalent to ARM's valid bit
1	R/W	Writable
2	U/S	User-accessible
3, 4, 7	PWT, PCD, PAT	Together index the PAT register (the x86 analogue of MAIR)
5	A (accessed)	Always set by hardware
6	D (dirty)	A real hardware dirty bit, set on first write
8	G (global)	Survives CR3 switches (ARM's nG = 0)
9–11, 52–58	Available	For OS use
12–51	Page frame number	Physical address
59–62	Protection key	Memory protection keys (PKU)

63	NX	No-execute

The two designs encode almost the same information; the main differences are ARM's split user/kernel execute-never bits, its permission encoding, and its use of DBM instead of a straightforward dirty bit.

How does the SoC interconnect decode physical addresses?

The interconnect is essentially a set of address comparators plus a switch. Each physical address range in the SoC is wired to a particular destination port, and the decoder picks the port whose range contains the address. It usually happens in layers: a coarse decision at the top-level fabric, then finer decisions in bridges further down.



1. The address map is decided at chip design time

Before any silicon exists, the SoC architects write down a **physical memory map**: which address ranges belong to DRAM, which to each peripheral, which to PCIe, and which are left unmapped. That map gets baked into the interconnect's RTL as decoder parameters.

The addresses in the diagram come from QEMU's ARM virt machine, which is a good real reference:

Physical range	Destination
0x0000_0000	Boot flash
0x0800_0000	GIC (interrupt controller)
0x0900_0000	PL011 UART
0x0901_0000	PL031 RTC
0x0903_0000	GPIO
0x0a00_0000	virtio-mmio devices
0x1000_0000	PCIe MMIO window
0x4000_0000 and up	DRAM

Software doesn't probe for this. It learns the map from the **device tree** (each node's reg property gives base and size) or from **ACPI tables**. On a running Linux system, /proc/iomem shows the result.

2. How a single decoder works

When the CPU issues a transaction on an AXI bus, the address travels on the address channel (ARADDR for reads, AWADDR for writes). The decoder compares it against every range **in parallel**, in a single cycle or two, and asserts a select signal for exactly one output port:

```
// conceptual top-level decoder
assign sel_ddr  = (addr >= 40'h00_4000_0000);
assign sel_pcie = (addr >= 40'h00_1000_0000) && (addr < 40'h00_3f00_0000);
assign sel_apb  = (addr[39:24] == 16'h0009);        // 0x0900_0000 – 0x09FF_FFFF
assign sel_def  = !(sel_ddr | sel_pcie | sel_apb | ...); // nothing matched
```

Designers usually make regions **power-of-two sized and aligned**, so instead of full magnitude comparisons the decoder only checks a few top address bits for equality, as sel_apb does above. This is fast and cheap in gates.

The transaction then passes through the crossbar switch to the selected port. The response (read data or write acknowledgment) comes back along the same path, matched to the right requester by the AXI transaction ID.

3. Decoding happens in layers

The top-level fabric only makes a coarse decision ("this is somewhere in the peripheral region") and forwards the transaction. Downstream components decode the remaining bits:

1. **Bridges.** An AXI-to-APB bridge receives everything in, say, 0x0900_0000–0x09FF_FFFF. It decodes the middle address bits and asserts the APB select line (PSELx) for one specific peripheral, giving each device a small slot (often 4 KB or 64 KB).
2. **Inside the peripheral.** The lowest bits select a register. For the PL011 UART, offset 0x000 is the data register and offset 0x018 is the flag register, so a write to 0x0900_0000 sends a character, while a read from 0x0900_0018 checks whether the transmit FIFO is full.
3. **Inside the DDR controller.** The controller takes the PA and decodes it again, this time into DRAM coordinates: channel, rank, bank group, bank, row and column. Which PA bits map to which coordinate is chosen to spread consecutive accesses across banks and channels, maximizing parallelism.

A practical gotcha comes from this layering: many decoders do **partial decoding**, ignoring some address bits to save logic. A small device in a 64 KB slot might only look at the low 12 bits, so its registers repeat (alias) every 4 KB throughout the slot.

4. What happens for an unmapped address

If no range matches, the transaction goes to a **default slave**, which exists only to respond with an AXI **DECERR** (decode error). A peripheral that exists but rejects an access (wrong size, powered off) returns **SLVERR** instead.

The CPU turns either response into an **external abort**. On arm64, a read typically produces a synchronous data abort, because the core is waiting for the data. A write is often "posted" (the core moves on before the response arrives), so the error shows up later as an asynchronous **SError**, which makes the offending instruction hard to pinpoint. This is why a buggy driver writing to a wrong MMIO address often produces a confusing SError Interrupt panic in Linux.

Without a default slave, the transaction would never receive a response and the bus could hang forever, so every well-designed fabric has one.

5. Large server-class SoCs: coherent mesh interconnects

High-core-count chips use a mesh network-on-chip (for example, Arm's CMN family) instead of a simple crossbar. The same idea applies but with more structure:

- Each requester (CPU cluster, I/O agent) has a **System Address Map (SAM)** that maps a PA to a destination **home node**.
- Memory addresses go to **fully coherent home nodes**, which manage cache coherence and own slices of the system-level cache. The PA is **hashed** across many home nodes so traffic spreads evenly over the mesh.
- I/O and device addresses go to **I/O home nodes**, which forward them to peripherals without coherence handling.
- Home nodes then forward memory requests to the memory controllers, again often **interleaved** across channels by address bits (for example, alternating every 256 bytes between channels) to multiply bandwidth.

6. The parts that can change at runtime

Most of the map is hardwired, but a few pieces are programmable:

- **PCIe windows and BARs.** The SoC reserves a fixed window (like 0x1000_0000 above), but which PCIe device appears where inside it is decided at boot when firmware or the OS enumerates the bus and programs each device's Base Address Registers. The root complex may also translate the CPU's PA into a different PCIe bus address using an outbound address translation unit.
- **Boot remapping.** Many SoCs alias the boot ROM at address 0x0 at reset (since the CPU starts executing there), then let firmware flip a remap register to put RAM or something else at that address.
- **Security filtering.** With TrustZone, the Non-Secure (NS) bit travels alongside the address, effectively creating separate secure and non-secure physical address spaces. A TrustZone address space controller (such as Arm's TZC-400) sits in front of the DDR controller and blocks non-secure masters from reaching regions reserved for secure firmware. Other firewalls do the same for peripherals.

Putting it together with the MMU

So the full chain for something like writel(c, uart_base) in a Linux driver is: the MMU translates the ioremap-ed VA to PA 0x0900_0000 with Device attributes, so the write bypasses the cache; the main interconnect sees top bits 0x09 and routes it to the APB bridge; the bridge asserts the UART's select line; the UART decodes offset 0x000 as its data register and pushes the byte into its transmit FIFO. The same instruction with a VA mapped to 0x4000_0000 and Normal attributes would instead go into the cache and, eventually, to the DDR controller.

What does Normal cacheable memory mean for DDR accesses?

"Normal cacheable" is the memory type the OS gives to RAM. It tells the CPU: *this memory behaves like plain storage, so you're free to cache it, prefetch it, reorder accesses to it, and combine them*. The result is that most loads and stores never reach DDR at all, and the ones that do reach it look very different from what the program actually executed.

What the Normal type permits

The memory type is a contract about side effects. Device memory says "every access matters, exactly as written"; Normal memory says "reading or writing has no side effects beyond storing data," which unlocks a set of optimizations:

Behavior	Normal memory	Device memory
Caching	Allowed (if marked cacheable)	Never
Speculative reads and hardware prefetch	Allowed	Forbidden
Reordering accesses	Allowed (weakly ordered)	Restricted (nR)
Merging several writes into one	Allowed	Restricted (nG)
Unaligned access	Allowed	Alignment fault
Exclusives/atomics	Fully supported (in shareable WB memory)	Often unsupported or faults

All of these are fine for RAM, because reading a DRAM location twice, or early, or never, changes nothing. They would be disastrous for a UART FIFO, where a speculative read would silently consume a received byte.

The cacheability attributes

"Cacheable" is itself configurable. Recall that the PTE's AttrIndx selects a byte in MAIR_EL1. For Normal memory, that byte is split into two halves:

MAIR byte = [7:4] outer attributes | [3:0] inner attributes
0xFF = outer write-back, read+write allocate | inner write-back, read+write allocate
0xBB = write-through, read+write allocate (both)
0x44 = non-cacheable (both)

Inner vs outer splits the cache hierarchy into two domains. "Inner" typically covers the CPU cluster's own caches (L1, L2, and often L3), and "outer" covers system-level caches beyond that. Where exactly the boundary falls is implementation-defined; for normal RAM, Linux simply marks both sides write-back.

The write policy controls when stores reach DDR:

- **Write-back:** a store updates only the cache line and marks it dirty. DDR is updated later, when the line is evicted. This is the standard choice for RAM.
- **Write-through:** every store updates both the cache and the next level, so DDR stays current but sees much more write traffic.
- **Non-cacheable:** no caching at all, but because it's still *Normal*, the CPU can merge adjacent writes into larger bursts. This is what Linux uses for framebuffers and some DMA buffers (the "write-combining" type).

Allocation hints say whether a miss should bring the line into the cache (read-allocate, write-allocate), and a "transient" hint suggests data that won't be reused and should be evicted early.

What DDR actually sees

With write-back caching, the DDR controller never sees individual loads and stores. It only sees whole **cache lines** (64 bytes on most ARM and x86 cores) moving in and out of the cache.

A load

1. LDR W0, [X1] reads 4 bytes.
2. The L1 cache is checked first, roughly a nanosecond. On a hit, nothing leaves the core.
3. On a miss, the request goes to L2, then L3 or the system cache.

4. Only if every level misses does the interconnect send a **64-byte line fill** to the DDR controller. This takes on the order of 100 ns.
5. The full line is installed in the cache, and the 4 bytes are returned to the register.

The 64-byte unit fits DDR well: a DDR4 burst of 8 transfers on a 64-bit channel moves exactly 64 bytes. The next 15 sequential 4-byte loads then hit in L1 for free, and hardware prefetchers often fetch the following lines before the program even asks for them.

A store

1. STR W0, [X1] writes 4 bytes.
2. If the line isn't cached, the core fetches it first (read-for-ownership), because the cache line must hold the other 60 bytes too.
3. The 4 bytes are written into the cached line, and the line is marked **dirty**.
4. DDR is **not** updated. It now holds stale data.
5. Only when the line is evicted (to make room for something else) is the whole 64-byte line written back to DDR.

If the program writes the same variable a million times, DDR may see a single writeback, or none if the line stays cached. Cores also detect streaming writes that cover entire lines (such as memset) and skip the pointless read in step 2.

Coherence: DDR is not the source of truth

Because dirty data lives in caches, DDR can be out of date. For multiple cores to share memory correctly, RAM is also marked **Inner Shareable** (the SH = 11 bits in the PTE). That brings in the hardware cache coherence protocol (MESI/MOESI-style, implemented with snooping or directories in the interconnect).

When core 1 reads a line that core 0 has dirty in its cache, the interconnect fetches it from core 0's cache rather than from DDR. To software, all cores appear to share a single consistent memory, even though DDR itself is frequently stale.

Coherence keeps each location consistent, but Normal memory is still **weakly ordered**: other cores may observe writes to *different* addresses in a different order than they were issued. Code that synchronizes across cores (locks, lock-free queues) must use barriers (DMB) or acquire/release instructions (LDAR/STLR), which is what C11 atomics and Linux's smp_* barriers compile to.

Where this bites: DMA

The coherence protocol covers the CPUs. A device doing DMA may not participate in it, and then it reads and writes DDR directly, seeing exactly the stale data described above.

- **CPU writes a buffer, device reads it** (e.g. a network packet to transmit): the data may still be dirty in the CPU cache, so the device reads old contents from DDR. The driver must **clean** the cache lines first (DC CVAC on arm64), forcing them out to DDR.
- **Device writes a buffer, CPU reads it** (e.g. a received packet): the CPU may hit old copies of those lines in its cache. The driver must **invalidate** the lines (DC IVAC) so the next load fetches fresh data from DDR.

Linux hides these operations behind the DMA API (dma_map_single(), dma_sync_single_for_cpu() and friends), which performs the right cache maintenance on non-coherent systems and does nothing on systems where it isn't needed. There are two alternatives:

1. **I/O-coherent interconnects** (for example, devices connected via ACE-Lite or CHI ports) let device accesses snoop the CPU caches, so no manual maintenance is needed.
2. **Non-cacheable buffers**: on non-coherent systems, dma_alloc_coherent() typically maps the buffer as Normal non-cacheable, so CPU accesses go straight to DDR.

One rule matters a lot here: the same physical memory should not be mapped as cacheable through one VA and non-cacheable through another at the same time. These **mismatched attributes** can let a stale or dirty cached copy coexist with direct DDR accesses and lead to silent data corruption.

Why this matters even before the OS runs

On arm64, when the stage-1 MMU is off (SCTLR_EL1.M = 0), data accesses are treated as **Device-nGnRnE**, even when they target DRAM. Early boot code therefore runs uncached and very slowly, and an unaligned access (which C compilers may generate for memcpy) faults. This is one of the reasons bootloaders and kernels set up page tables marking RAM as Normal write-back as one of their first tasks.

Why must MMIO be mapped as Device memory? Explain nGnRE etc.

Device memory exists because MMIO registers aren't storage. Reading or writing them *does something*: it pops a FIFO, clears an interrupt, starts a DMA transfer. Every optimization the CPU is allowed to apply to Normal memory assumes accesses have no side effects, and each of those optimizations breaks a real device in a specific way.

What goes wrong if MMIO is mapped as Normal

Take a typical UART and network controller and walk through what each Normal-memory optimization would do.

Caching. A driver polls a status register in a loop, waiting for "transmit FIFO empty". The first read fills a cache line, and every later read hits the cache, returning the same stale value forever while the hardware has long since changed it. Writes are worse: with write-back caching, a write to the UART data register sits dirty in the cache and the character is never sent, or it's sent much later when the line happens to be evicted, possibly along with 60 other bytes of "register" writes the driver never made.

Speculative reads and prefetch. Many registers have read side effects. Reading a UART's receive data register removes the byte from the FIFO; reading some interrupt status registers clears the pending bits. If the CPU speculatively loads such a register down a branch that's later discarded, the side effect has already happened: a received byte or an interrupt is silently lost. Hardware prefetchers would do the same to neighboring registers the program never touched.

Merging (gathering). A driver writes two bytes to a FIFO register: writeb('A', data); writeb('B', data);. With Normal memory, the CPU may merge these into one write (the second simply overwrites the first in the store buffer), so the device sees only 'B'. Or it may combine writes to *adjacent* registers into one wider bus transaction, when the device only accepts accesses of a specific width.

Reordering. A driver programs a DMA engine by writing the source address, then the length, then setting the START bit. If the START write overtakes the others, the engine starts with garbage parameters.

Access size changes. Many peripherals only accept 32-bit accesses to their registers. Normal memory allows the CPU and interconnect to split, widen, or realign accesses (for example, splitting an unaligned access), which some devices reject with a bus error or misinterpret.

Device memory forbids all of this. Accesses are never cached, never speculative (only executed instructions touch the device), and unaligned accesses fault rather than being silently split. The remaining question is *how strict* the ordering and merging rules should be, and that's what the G, R and E letters encode.

Decoding the letters

Each Device type is named by three attributes, where a leading n means "not permitted":

G: Gathering

nG (no gathering): the number and size of accesses that appear on the bus exactly match what the program executed. Two writeb() calls become two separate byte writes. A read is never satisfied from an earlier write still in the store buffer; it always goes to the device.

G (gathering allowed): multiple accesses may be merged into one transaction, either several writes to the same location (keeping only the last) or writes to adjacent locations combined into a wider burst.

R: Reordering

nR (no reordering): accesses to the same peripheral arrive in program order. (The architecture defines "same peripheral" as an implementation-defined block size, so this guarantee covers registers within one device, not across unrelated devices.)

R (reordering allowed): accesses to the device may arrive in a different order than issued.

E: Early write acknowledgment

This one is subtler. When a CPU issues a write, it waits for a write *response* before considering the write complete. The question is who sends that response.

nE (no early acknowledgment): the response must come from the peripheral itself, so "complete" really means the device received the write.

E (early acknowledgment allowed): an intermediate buffer in the interconnect, such as a bridge's write buffer, may acknowledge the write immediately and forward it later. This is called a **posted write**. It's faster, because the CPU doesn't wait for the full round trip, but the CPU may believe a write has landed when it's still in transit.

The four Device types

Type	MAIR byte	Gather	Reorder	Early ack	Typical use
Device-nGnRnE	0x00	No	No	No	Strictest: PCI config space, some SoC buses (e.g. Apple M1)
Device-nGnRE	0x04	No	No	Yes	**Default for MMIO** (Linux ioremap)
Device-nGRE	0x08	No	Yes	Yes	Devices tolerant of reordering
Device-GRE	0x0C	Yes	Yes	Yes	Rare; weakest Device type

Even Device-GRE still forbids caching and speculative reads, so it remains a Device type. The step beyond it is Normal non-cacheable (0x44), which Linux uses for ioremap_wc() on framebuffers: there, merging writes into large bursts is exactly what you want, and reading pixel memory has no side effects.

For readers familiar with ARMv7, nGnRnE corresponds to the old "Strongly-ordered" type and nGnRE to the old "Device" type. On x86, MMIO is normally mapped **UC** (uncacheable), which behaves roughly like nGnRnE, while **WC** (write-combining) corresponds to Normal non-cacheable.

Why nGnRE is the usual default

nG and nR are needed by almost every peripheral, for the FIFO and DMA-programming reasons above. Early acknowledgment, by contrast, is usually harmless: posted writes still arrive at the device in order, and the speed gain on high-latency bus paths is worthwhile.

Some situations do require nE:

- **PCIe configuration space**: the PCIe specification requires configuration writes to be non-posted, so Linux maps it as nGnRnE.
- **Certain SoCs**: Apple's M1 family requires non-posted writes for its internal peripherals; nGnRE mappings there cause SErrors. Linux added ioremap_np() ("non-posted") in version 5.13 largely for this reason.
- **Code that must know a write has reached the device**, for example before cutting the device's clock or power.

Even nE has a limit: the device *receiving* a write isn't the same as the device having *acted on it*. If a driver writes a register to clear an interrupt and immediately re-enables interrupts, the interrupt line may still be asserted for a few cycles. The standard fix is to **read back** a register from the same device. Since the read can't overtake the earlier write (nR), the read's return guarantees the write was processed. This read-back idiom is common in PCI drivers, where writes are posted all the way down the bus.

Device ordering is not ordering against RAM

The nR guarantee only covers accesses to the same device. It says nothing about how Device accesses are ordered relative to **Normal memory** accesses, and that gap is behind a classic DMA bug:

```
desc->addr = buf_dma;     // 1. write DMA descriptor in RAM (Normal memory)
desc->len  = len;
writel(1, regs + DOORBELL);  // 2. tell the device to fetch the descriptor (Device memory)
```

Without a barrier, the doorbell write may reach the device before the descriptor writes are visible in memory, so the device fetches a half-written descriptor. This is why Linux's writel() on arm64 includes a barrier (DMB OSHST) **before** the store, and readl() includes one **after** the load (so later reads of a DMA buffer aren't satisfied before the status register says the DMA finished). The _relaxed variants (writel_relaxed(), readl_relaxed()) drop these barriers and are only safe when there's no DMA interaction to order against.

For "wait until the write has completed" semantics, such as before WFI or before assuming a device has been reset, a DSB is needed, since DMB only orders accesses relative to each other.

Other things Linux sets for MMIO mappings

Linux marks every MMIO mapping **execute-never** (PXN/UXN), since an instruction fetch from a device region, including a speculative one, could trigger read side effects. The Device type also means unaligned accesses, DC ZVA (zero-a-cache-line), and most exclusive or atomic instructions either fault or behave in implementation-defined ways, which is why drivers stick to plain, naturally aligned readl/writel-style accessors.

how interrupts are handled in Cortex M

On ARM Cortex-M, interrupt handling is done mostly in hardware. The core saves context, finds the handler, and restores state on its own, so interrupt handlers can be ordinary C functions with no special keywords or assembly wrappers. The process has four parts.

1. The building blocks

Exceptions vs. interrupts. ARM calls everything an "exception." Numbers 1–15 are *system exceptions*: Reset, NMI, HardFault, MemManage, BusFault, UsageFault, SVCall, PendSV, and SysTick. Numbers 16 and up are *external interrupts* (IRQs) from peripherals such as UART, timers, and GPIO. IRQ0 is exception 16.

The vector table. This is an array of 32-bit addresses, normally at address 0x00000000. Entry 0 is the initial Main Stack Pointer value. Entry 1 is the Reset handler, and the rest are handler addresses indexed by exception number. On M3/M4/M7 (and optionally M0+/M23/M33), the VTOR register lets you move the table, for example into RAM or behind a bootloader. Every handler address must have bit 0 set (the Thumb bit). Toolchains handle this automatically.

The NVIC (Nested Vectored Interrupt Controller). This is tightly coupled to the core and manages each IRQ's state:

- **Enable/disable** via the ISER/ICER registers
- **Pending** status via ISPR/ICPR: the interrupt has been requested but not yet serviced
- **Active** status: the handler is currently running or has been preempted
- **Priority** via the IPR registers

2. Priorities and nesting

A lower number means higher urgency. Reset (-3), NMI (-2), and HardFault (-1) have fixed priorities that cannot be changed. Everything else is configurable. The Cortex-M0/M0+ implements 2 priority bits (4 levels), while M3/M4/M7 implement 3–8 bits depending on the silicon vendor. STM32 parts typically use 4 bits.

On M3 and above, the PRIGROUP field in SCB->AIRCR splits each priority value into two parts:

- **Preemption priority (group priority)** decides whether a new interrupt can interrupt a running handler.

- **Subpriority** only decides which of several pending interrupts goes first. It never causes preemption.

If two pending interrupts have identical priority, the one with the lower exception number wins.

Three mask registers let software block interrupts:

- **PRIMASK** blocks everything except NMI and HardFault. This is what __disable_irq() sets.
- **FAULTMASK** additionally blocks HardFault. It is not available on M0.
- **BASEPRI** blocks only interrupts at or below a chosen priority level. RTOSes use it for critical sections that still let high-priority interrupts through. It is not available on M0.

3. What happens when an interrupt fires

Step 1: Stacking. The hardware pushes 8 registers onto the current stack (MSP or PSP): R0–R3, R12, LR, PC, and xPSR. These are exactly the "caller-saved" registers under the ARM calling convention (AAPCS). Any registers the handler itself uses (R4–R11) are saved by the compiler-generated prologue, the same as in any C function. That is why handlers need no __interrupt attribute. If the FPU is active (M4F/M7), an extended frame also covers S0–S15 and FPSCR. *Lazy stacking* reserves that space but only writes the FP registers if the handler actually uses the FPU.

Step 2: Vector fetch. In parallel with stacking, the core reads the handler address from the vector table over a separate bus path.

Step 3: State update.

- The core switches to **Handler mode**, which is always privileged and always uses the **MSP**.
- IPSR is set to the exception number.
- The interrupt's state changes from pending to active.
- LR is loaded with a special **EXC_RETURN** value such as 0xFFFFFFF9 or 0xFFFFFFFD. This value records which stack to return to, which mode, and whether an FP frame was saved.

Step 4: Execute the handler. Total entry latency is about **12 cycles** on M3/M4 and about 16 on M0, assuming zero-wait-state memory.

Step 5: Return. The handler ends with a normal BX LR or a POP {..., PC}. Because the value loaded into PC is an EXC_RETURN pattern rather than a real address, the hardware recognizes it as an exception return. It then unstacks the registers and resumes the interrupted code.

4. Latency optimizations

Tail-chaining. If another interrupt is pending when a handler finishes, the core skips the unstack-and-restack cycle and jumps straight to the next handler. This takes about 6 cycles instead of roughly 24.

Late arrival. If a higher-priority interrupt arrives while the core is still stacking for a lower-priority one, the core reuses that stacking and vectors to the higher-priority handler first.

Pop preemption. If a new interrupt arrives during unstacking, the core abandons the unstack and goes directly into the new handler.

A typical setup (CMSIS)

```
void TIM2_IRQHandler(void)          // name must match the vector table entry
{
  if (TIM2->SR & TIM_SR_UIF) {
    TIM2->SR &= ~TIM_SR_UIF;     // clear the PERIPHERAL flag, or it re-fires
immediately
    // ... do work ...
  }
}

int main(void)
{
  NVIC_SetPriorityGrouping(3);     // e.g. 4 bits preemption, 0 bits sub (STM32 style)
  NVIC_SetPriority(TIM2_IRQn, 5);
  NVIC_EnableIRQ(TIM2_IRQn);
  // configure TIM2 to generate update interrupts...
}
```

Practical gotchas

Clear the peripheral flag. The NVIC pending bit clears automatically on entry, but the peripheral's own flag does not. If you don't clear it, the interrupt fires again immediately and the handler appears to run forever.

Clearing the flag too late. Because of write buffering on M3/M4/M7, clearing the flag as the very last instruction can cause a spurious re-entry. Clear it early in the handler, or add a __DSB().

Volatile shared variables. Variables shared between a handler and main code must be volatile, and any read-modify-write on them needs protection with a critical section.

PendSV and SVCall. RTOSes use PendSV at the lowest priority to perform context switches, and SVCall for system calls. This design means a task switch never preempts a real interrupt handler.

HardFault escalation. A fault occurring while that fault type is disabled, or a fault inside a handler at the same or higher priority, escalates to HardFault. This is a common cause of mysterious HardFaults.

how interrupts are handled in Cortex A

On Cortex-A, interrupt handling is split between two pieces of hardware: the **Generic Interrupt Controller (GIC)**, which collects, prioritizes and routes interrupts, and the **core**, which takes an exception and jumps to software. Unlike Cortex-M, the core does almost nothing automatically. It doesn't stack registers or vector per interrupt, so software does most of the work.

1. The GIC: getting the interrupt to the core

Peripherals raise interrupt lines into the GIC, which classifies them by ID:

Type	ID range	Meaning
SGI (Software Generated)	0–15	Inter-processor interrupts, triggered by writing a GIC register
PPI (Private Peripheral)	16–31	Private to one core, e.g. the generic timer or PMU

SPI (Shared Peripheral)	32–1019	Shared peripherals (UART, Ethernet, ...), routable to any core
LPI (Locality-specific)	8192+	Message-based, GICv3/v4 only, used with PCIe MSI via the ITS

The GIC has three main parts. The **Distributor** handles enabling, priority, and target core or affinity for SPIs. The **Redistributor** (GICv3+) does the same per core for SGIs, PPIs and LPIs. The **CPU interface** signals the core and handles acknowledge and end-of-interrupt.

The GIC picks the highest-priority pending interrupt that beats the core's priority mask (ICC_PMR_EL1 / GICC_PMR) and current running priority. It then asserts either the **IRQ** or the **FIQ** line to the core. Which one depends on the interrupt's group (Group 0 vs. Group 1) and the core's security state. Typically Group 0 goes to FIQ for secure firmware, and Group 1 non-secure goes to IRQ for the OS.

2. What the core does in hardware

ARMv8-A (AArch64), e.g. Cortex-A53/A72/A78

When IRQ is asserted and not masked by PSTATE.I:

1. The target Exception Level is chosen. Routing is controlled by SCR_EL3.IRQ/FIQ (route to EL3) and HCR_EL2.IMO/FMO (route to EL2). Otherwise the interrupt goes to EL1.
2. The current PSTATE is saved into SPSR_ELx.
3. The return address is saved into ELR_ELx.
4. PSTATE.DAIF is set, so all of Debug, SError, IRQ and FIQ are masked, and the stack pointer switches to SP_ELx.
5. The PC jumps to VBAR_ELx plus an offset.

The vector table has 16 entries, each 0x80 bytes, laid out like this:

Source of exception	Sync	IRQ	FIQ	SError
Current EL, using SP_EL0	0x000	0x080	0x100	0x180
Current EL, using SP_ELx	0x200	**0x280**	0x300	0x380
Lower EL, AArch64	0x400	**0x480**	0x500	0x580
Lower EL, AArch32	0x600	0x680	0x700	0x780

So a kernel interrupted while in kernel mode lands at VBAR_EL1 + 0x280. One interrupted while running a user app lands at +0x480.

ARMv7-A (AArch32), e.g. Cortex-A7/A9/A15

The idea is the same, but it uses processor modes instead of exception levels:

1. CPSR is copied to SPSR_irq.
2. The return address (+4) is copied to LR_irq.
3. The mode switches to IRQ mode, which has its own banked SP and LR (FIQ mode banks R8–R12 too, which is why it's "fast").
4. CPSR.I is set, and for FIQ, CPSR.F as well.
5. The PC jumps to the vector base plus 0x18 for IRQ or 0x1C for FIQ.

3. What software does (the handler)

A typical AArch64 flow with GICv3 looks like this:

```
irq_el1h:                   // VBAR_EL1 + 0x280
  sub   sp, sp, #(34*8)
  stp   x0, x1,  [sp, #0]     // save general-purpose regs
  ...                   // (x2–x30)
  mrs   x0, elr_el1
  mrs   x1, spsr_el1
  stp   x0, x1, [sp, #(32*8)]  // save return state (needed if nesting)

  mrs   x0, ICC_IAR1_EL1      // ACKNOWLEDGE: read INTID, marks it active
  cmp   x0, #1020           // 1020-1023 = spurious
  b.hs  1f
  bl    dispatch_irq        // call the driver's handler for this INTID
  msr   ICC_EOIR1_EL1, x0     // END OF INTERRUPT: drop running priority
1:
  ldp   x0, x1, [sp, #(32*8)]
  msr   elr_el1, x0
  msr   spsr_el1, x1
  ...                   // restore regs
  eret                    // restores PSTATE from SPSR, PC from ELR
```

The key steps are:

1. **Save context.** Hardware saved only the PC and PSTATE, so the handler must save the general-purpose registers.

2. **Acknowledge.** Reading ICC_IAR1_EL1 (or the memory-mapped GICC_IAR on GICv2) returns the interrupt ID. It also moves that interrupt from pending to active and raises the CPU's running priority.
3. **Dispatch** to the device's handler, which clears the interrupt at the source peripheral.
4. **End of interrupt.** Writing ICC_EOIR1_EL1 drops the running priority. If split EOI mode is enabled, as hypervisors use, a separate write to ICC_DIR_EL1 deactivates it.
5. **Return** with ERET. On ARMv7 the return is SUBS PC, LR, #4 or RFE.

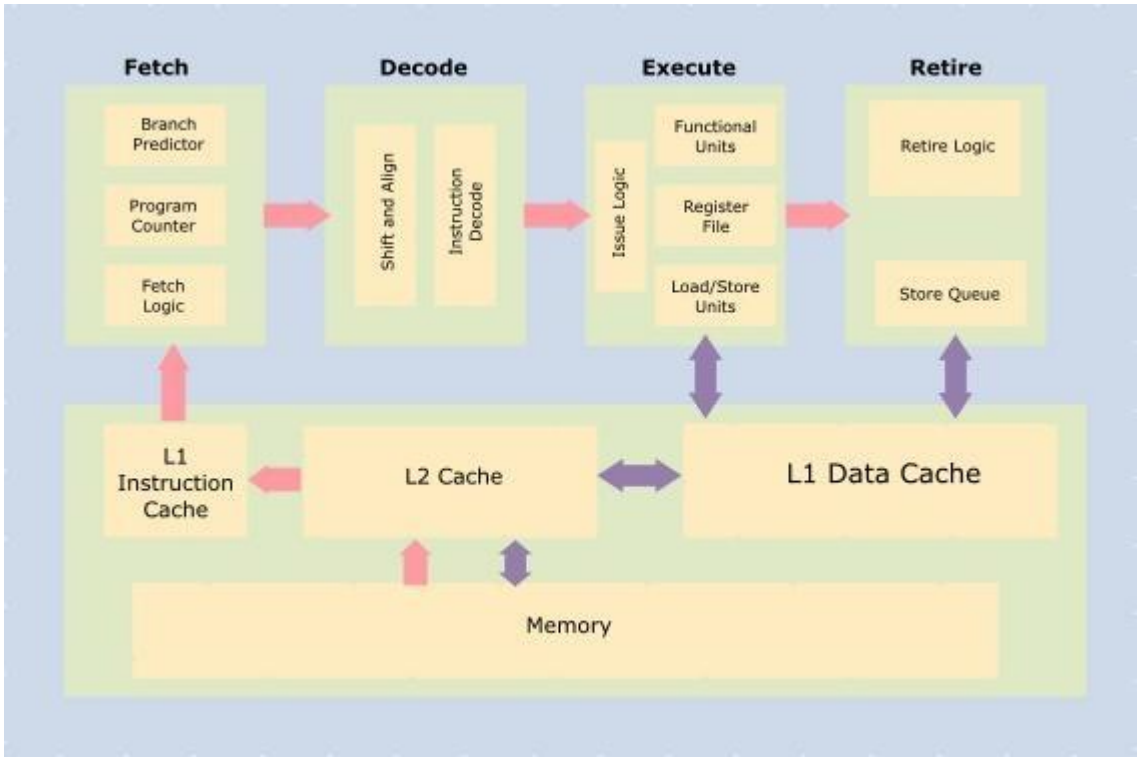
4. Nesting and priorities

Because entry masks IRQ/FIQ, interrupts don't nest by default. To allow nesting, the handler saves ELR/SPSR (they'd be overwritten), then unmasks with msr daifclr, #2. After that, the GIC only forwards interrupts with higher priority than the one currently active. Linux generally avoids nesting and instead defers work to softirqs and threaded IRQs. On GICv3 it uses ICC_PMR_EL1 priority masking to implement pseudo-NMIs.

5. Quick contrast with Cortex-M

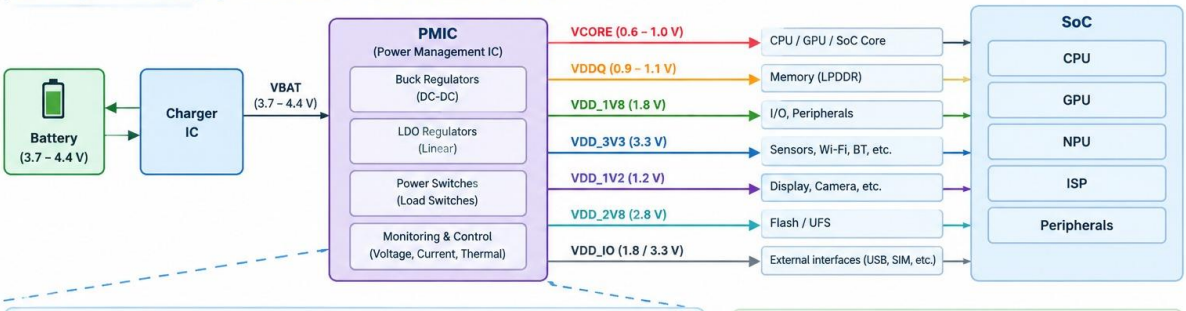
Cortex-M's NVIC automatically stacks registers, vectors directly to a per-interrupt handler, and supports tail-chaining. Cortex-A gives you a single IRQ entry point and expects software to ask the GIC "which one was it?" This adds a bit of latency but supports SMP, virtualization (GICv3/v4 virtual interrupts) and TrustZone routing.

In Linux, the chain is: vector entry (el1h_64_irq in entry.S), then handle_arch_irq, then gic_handle_irq(), then generic_handle_domain_irq(), then your driver's request_irq() handler.

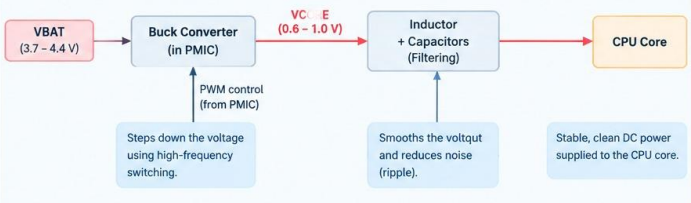


How Power Rails Supply Power (Android/SoC Perspective)

Power rails are regulated voltage lines that deliver the required voltage and current to different blocks in thie SoC and other components. They are generated from the battery (or charger) through a series of power management ICs (PMICs), regulators and switches, and then distributed across the board.



Example: VCORE Rail (CPU Core Power)



Key Points

1. **Battery/Charger** – Provides the main input (3.7–4.4 V).
2. **PMIC** – Generates multiple rails using buck/LDO regulators, power switches and controls sequencing.
3. **Power Rails** – Each rail has a specific voltage and current for different parts of the system.
4. **Distribution** – Rails are routed through PCB traces and connectors to the SoC and other components.
5. **Additional features** – PMIC handles power sequencing, protection (over-current, over-voltage, thermal) and monitoring.

Step-by-Step Flow



Different SoCs and devices may have slightly different rail voltages and architectures, but the principle remains the same.